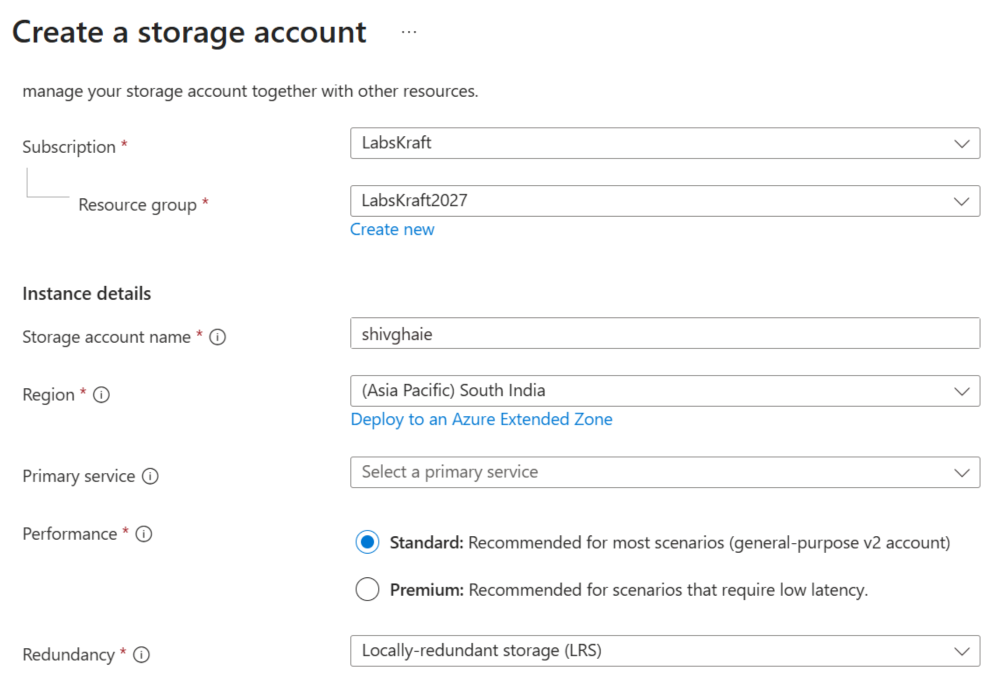
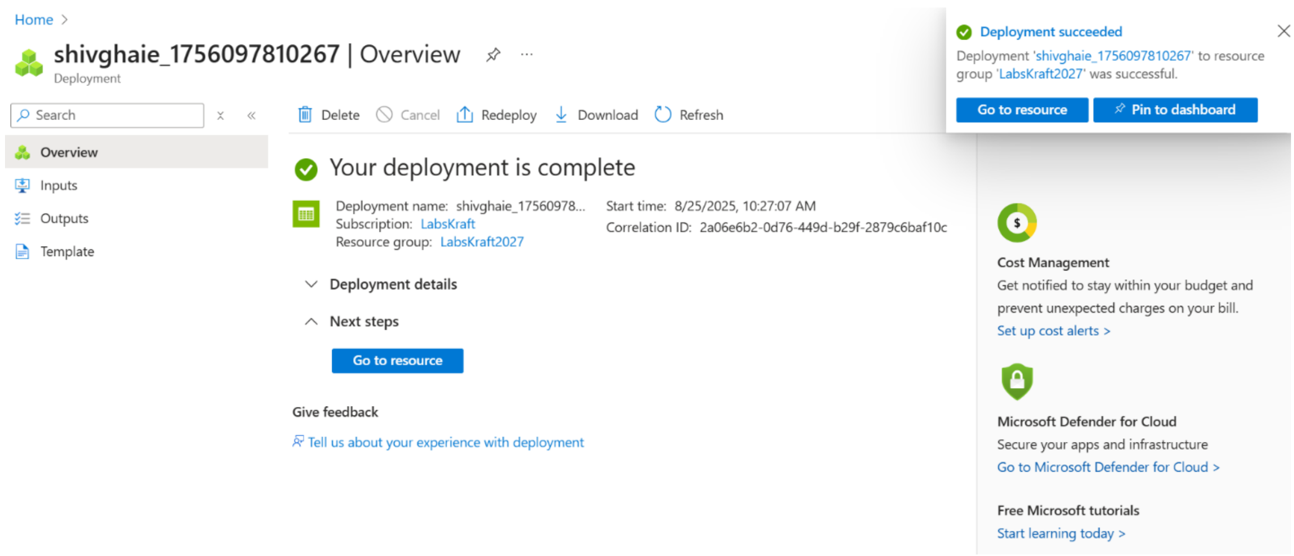
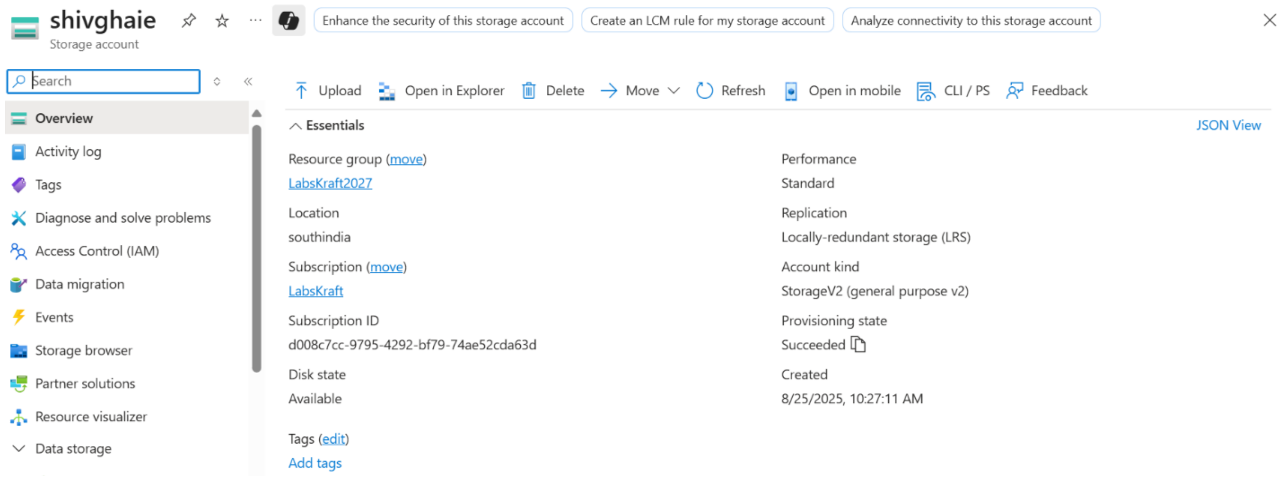
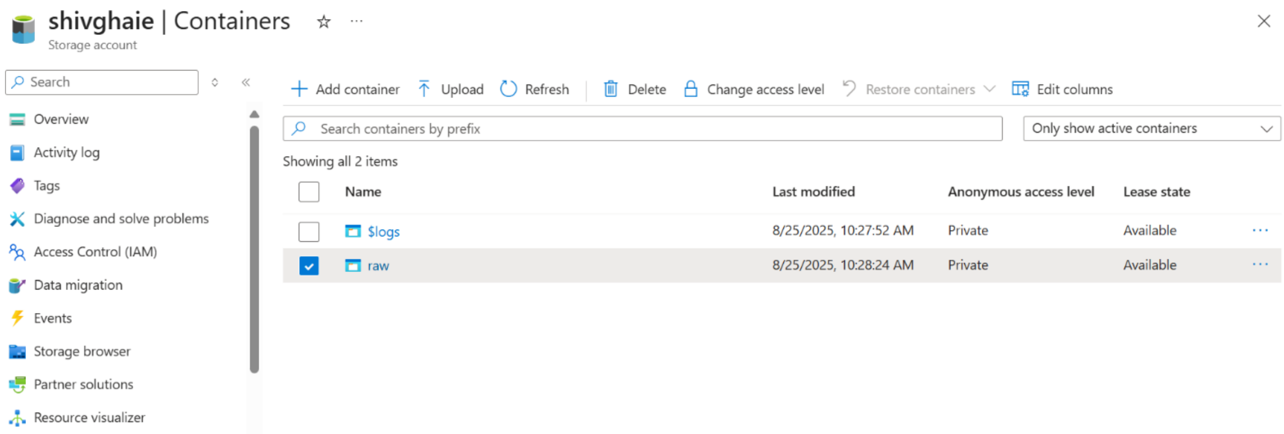
Shivankur Ghaie
Capstone Project - 1

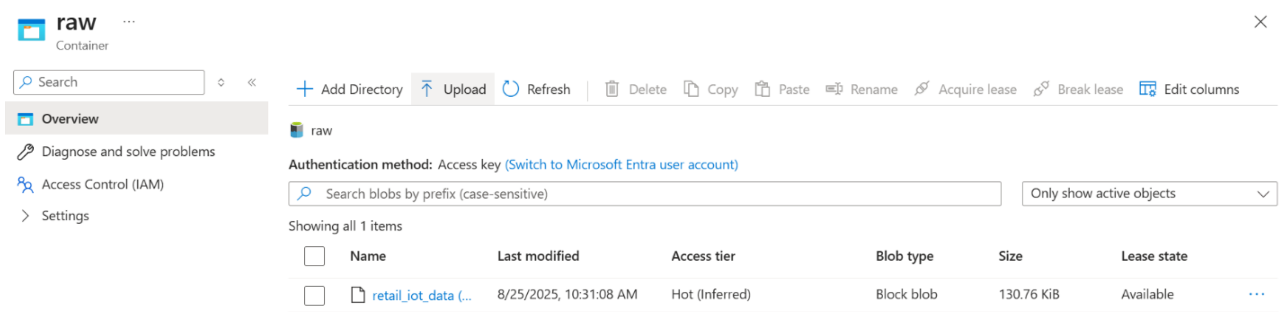
1. Creation of storage account:



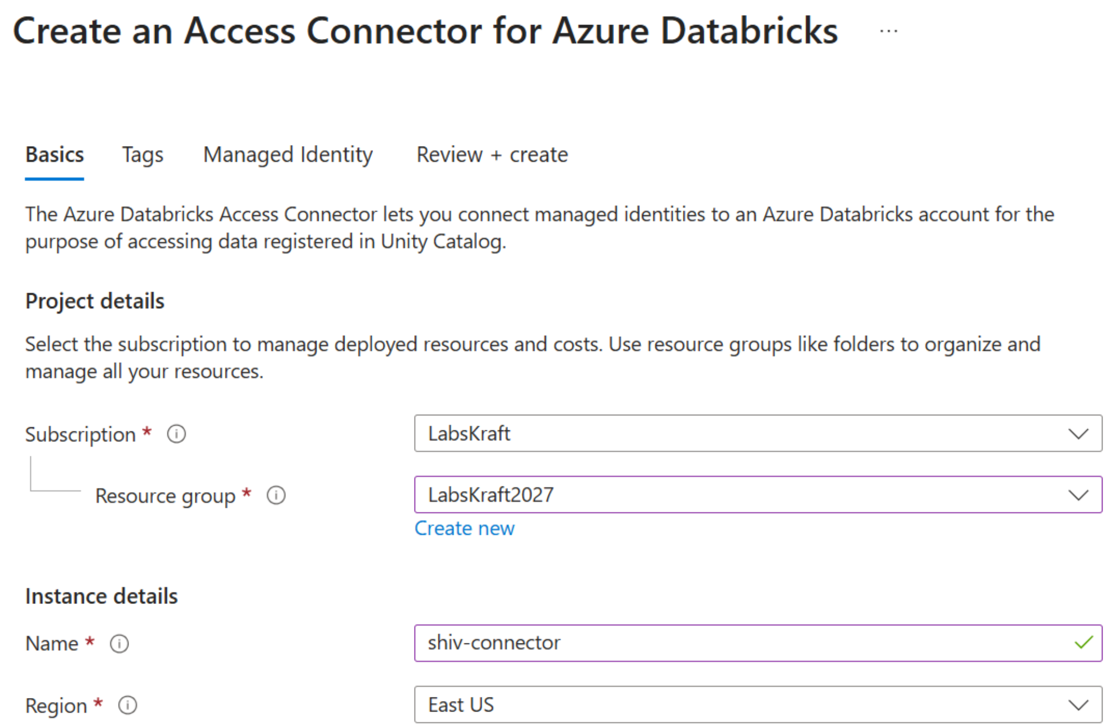


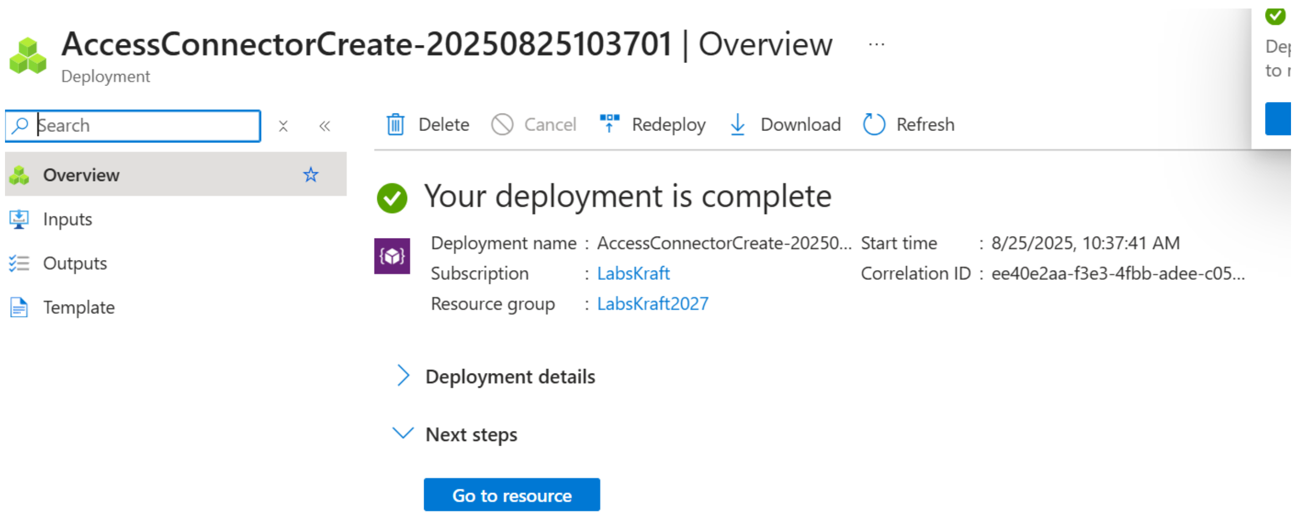

shivghaie
Storage account
Enhance the security of this storage account
Create an LCM rule for my storage account
Analyze connectivity to this storage account
Search
Overview
Activity log
Tags
Diagnose and solve problems
Access Control (IAM)
Data migration
Events
Storage browser
Partner solutions
Resource visualizer
Data storage
Upload
Open in Explorer
Delete
Move
Refresh
Open in mobile
CLI / PS
Feedback
Essentials
JSON View
Resource group (move)
LabsKraft2027
Location
southindia
Subscription (move)
LabsKraft
Subscription ID
d008c7cc-9795-4292-bf79-74ae52cda63d
Disk state
Available
Tags (edit)
Add tags
Performance
Standard
Replication
Locally-redundant storage (LRS)
Account kind
StorageV2 (general purpose v2)
Provisioning state
Succeeded
Created
8/25/2025, 10:27:11 AM


shivghaie | Containers
Storage account
Search
Overview
Activity log
Tags
Diagnose and solve problems
Access Control (IAM)
Data migration
Events
Storage browser
Partner solutions
Resource visualizer
Add container
Upload
Refresh
Delete
Change access level
Restore containers
Edit columns
Search containers by prefix
Only show active containers
Showing all 2 items
Name
Last modified
Anonymous access level
Lease state
$logs
8/25/2025, 10:27:52 AM
Private
Available
raw
8/25/2025, 10:28:24 AM
Private
Available

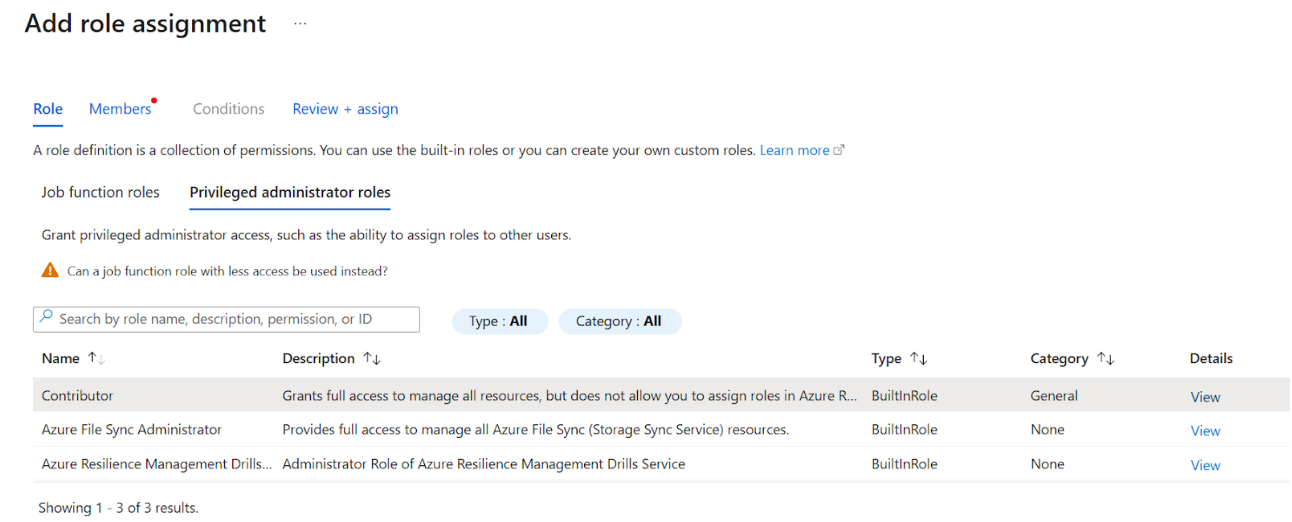

raw
Container
Search
Overview
Diagnose and solve problems
Access Control (IAM)
Settings
Add Directory
Upload
Refresh
Delete
Copy
Paste
Rename
Acquire lease
Break lease
Edit columns
raw
Authentication method: Access key (Switch to Microsoft Entra user account)
Search blobs by prefix (case-sensitive)
Only show active objects
Showing all 1 items
Name
Last modified
Access tier
Blob type
Size
Lease state
retail_iot_data (...
8/25/2025, 10:31:08 AM
Hot (Inferred)
Block blob
130.76 KiB
Available

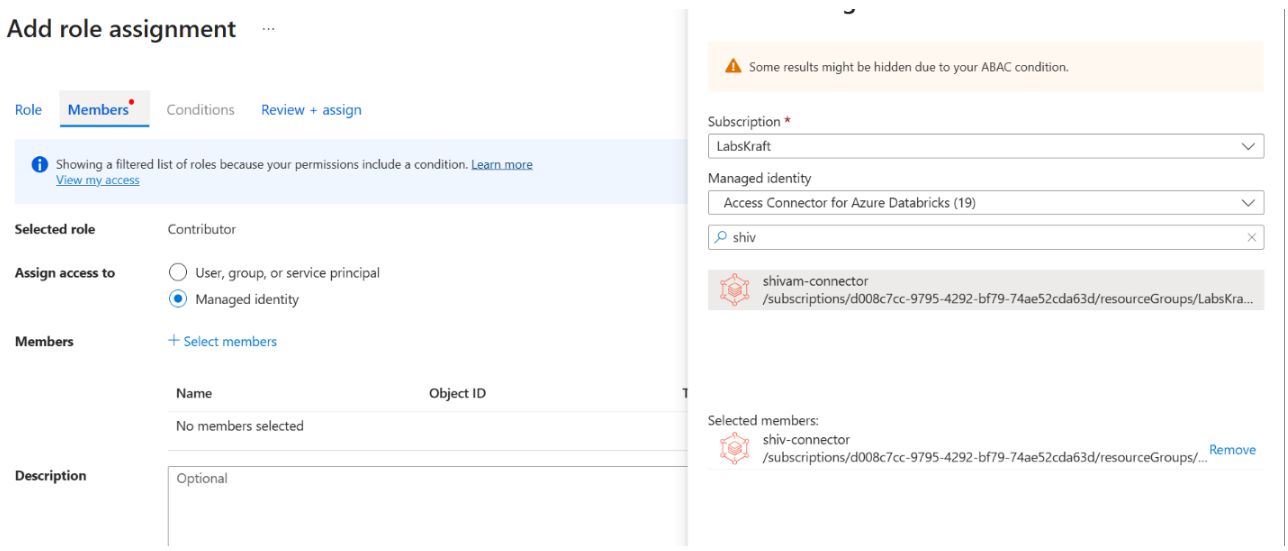
2. Creation of access connector:

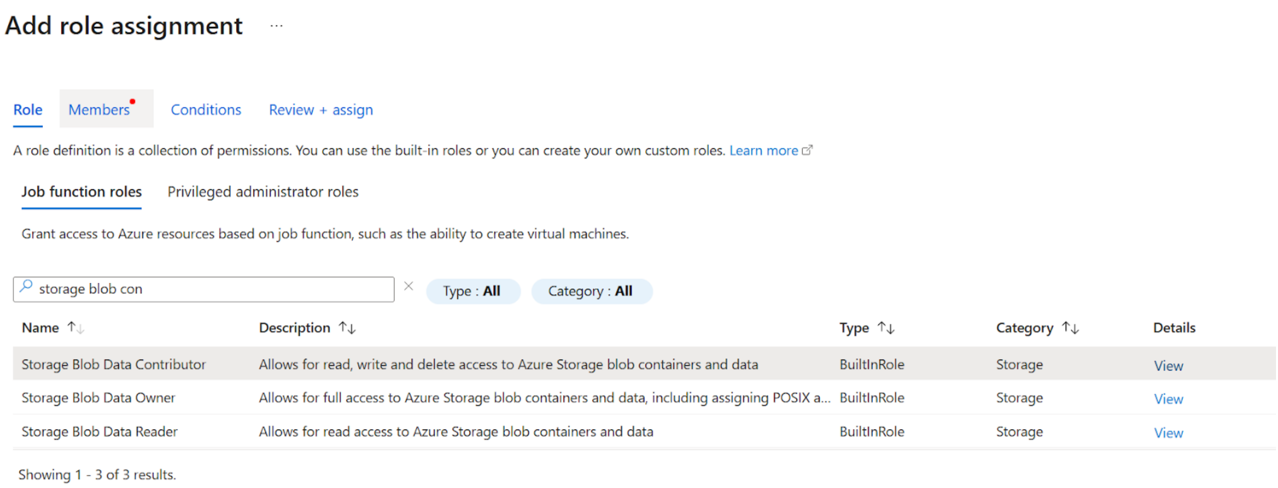




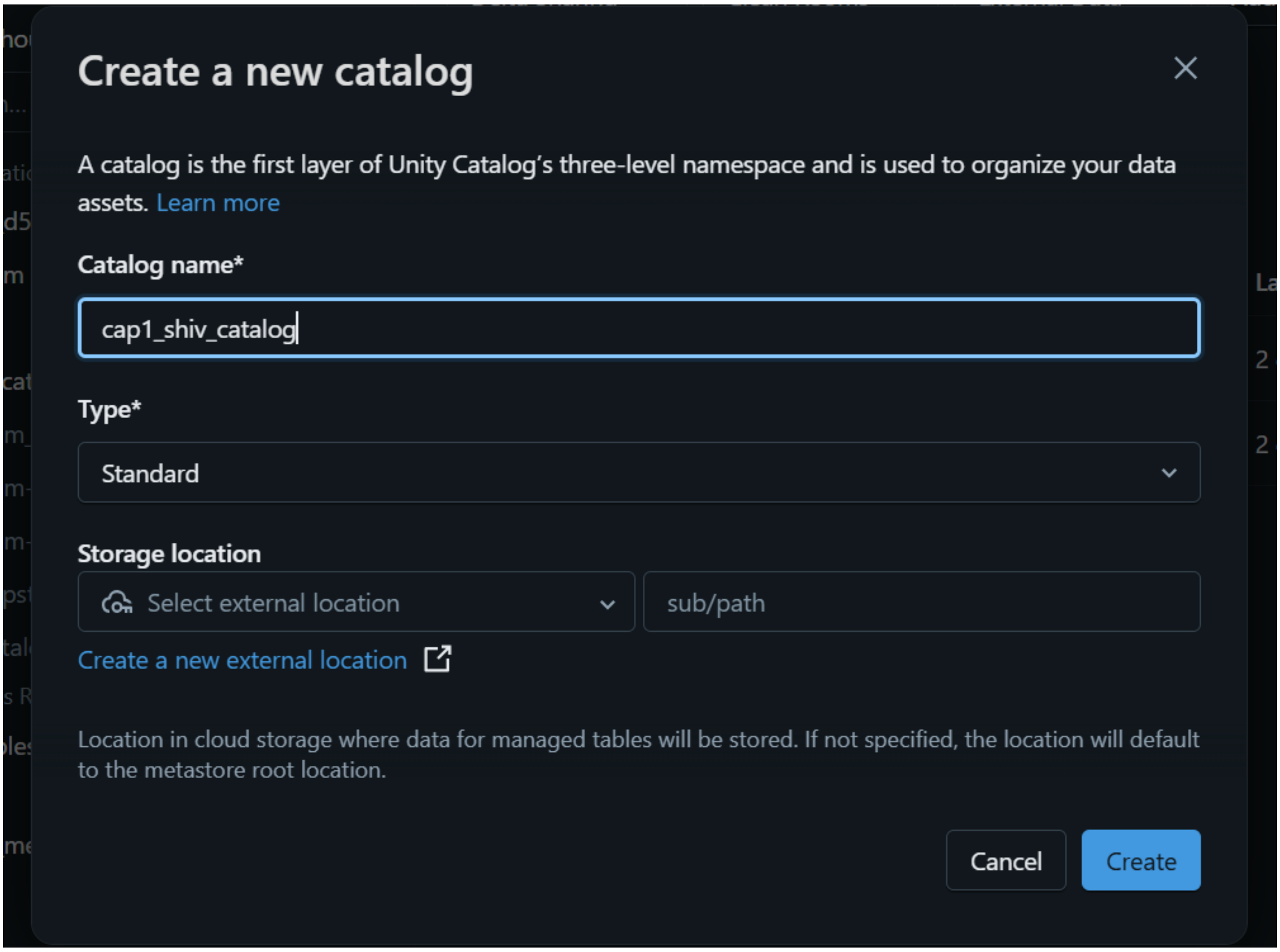
3. Giving access connector permission for storage:

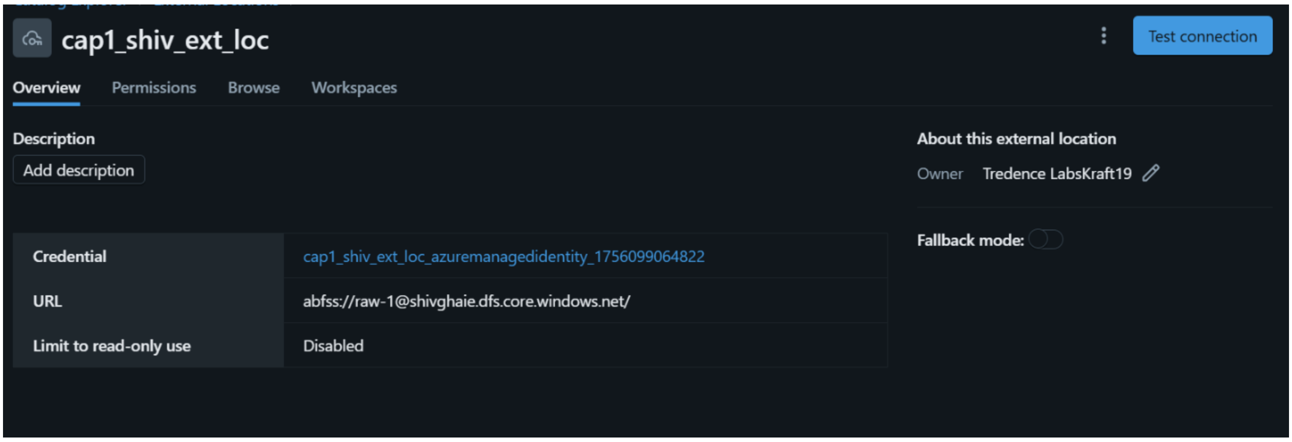


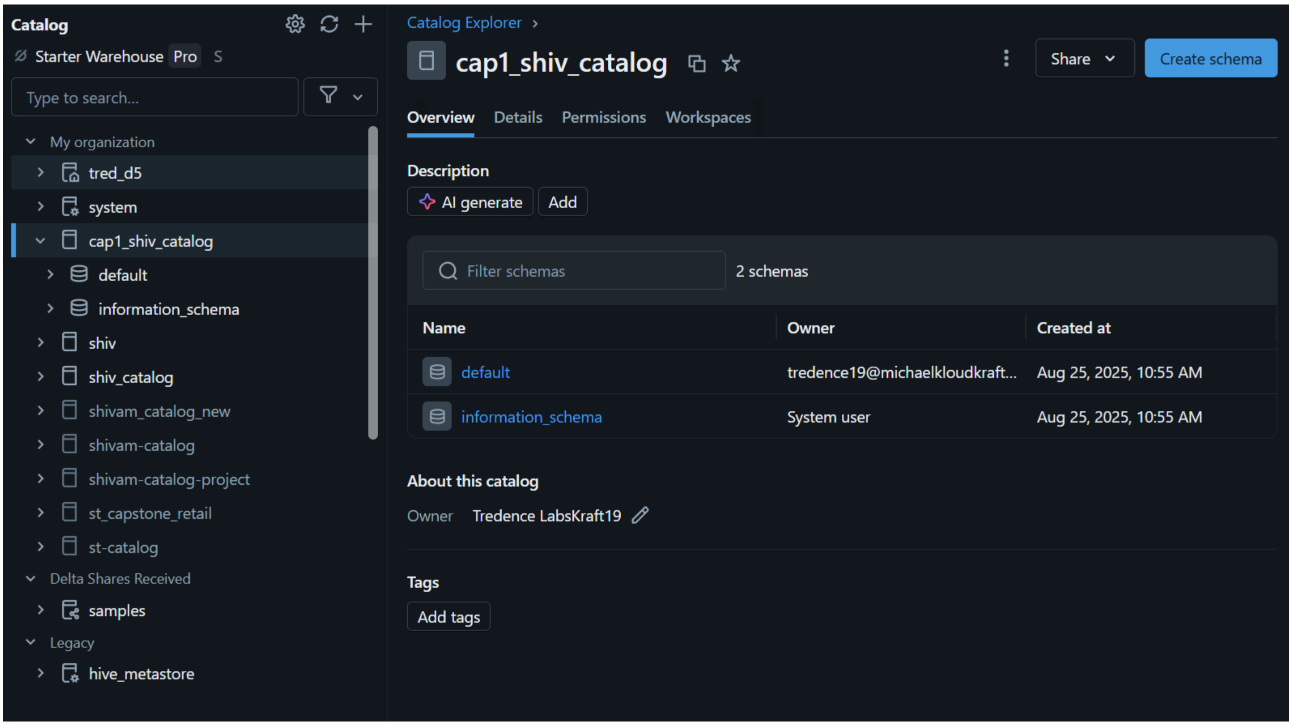




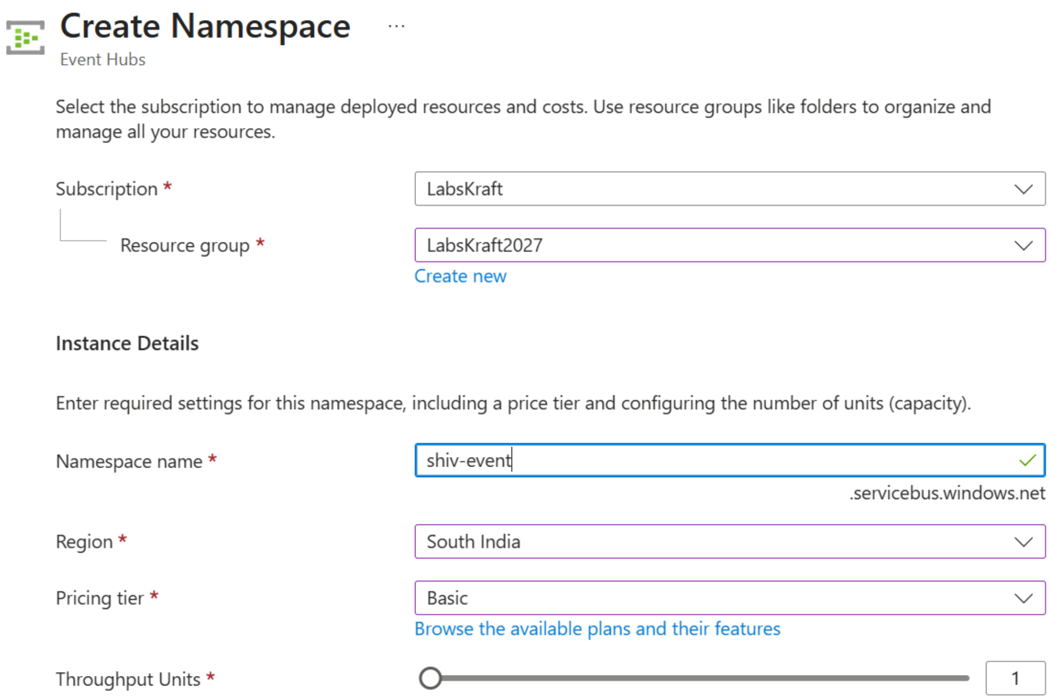
4. Creation of Unity Catalog:

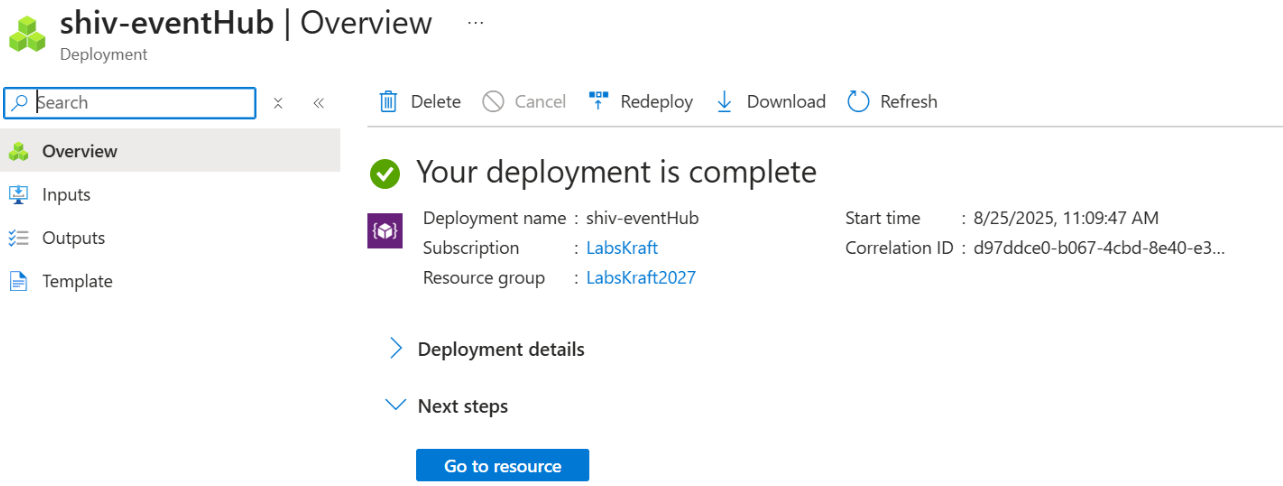


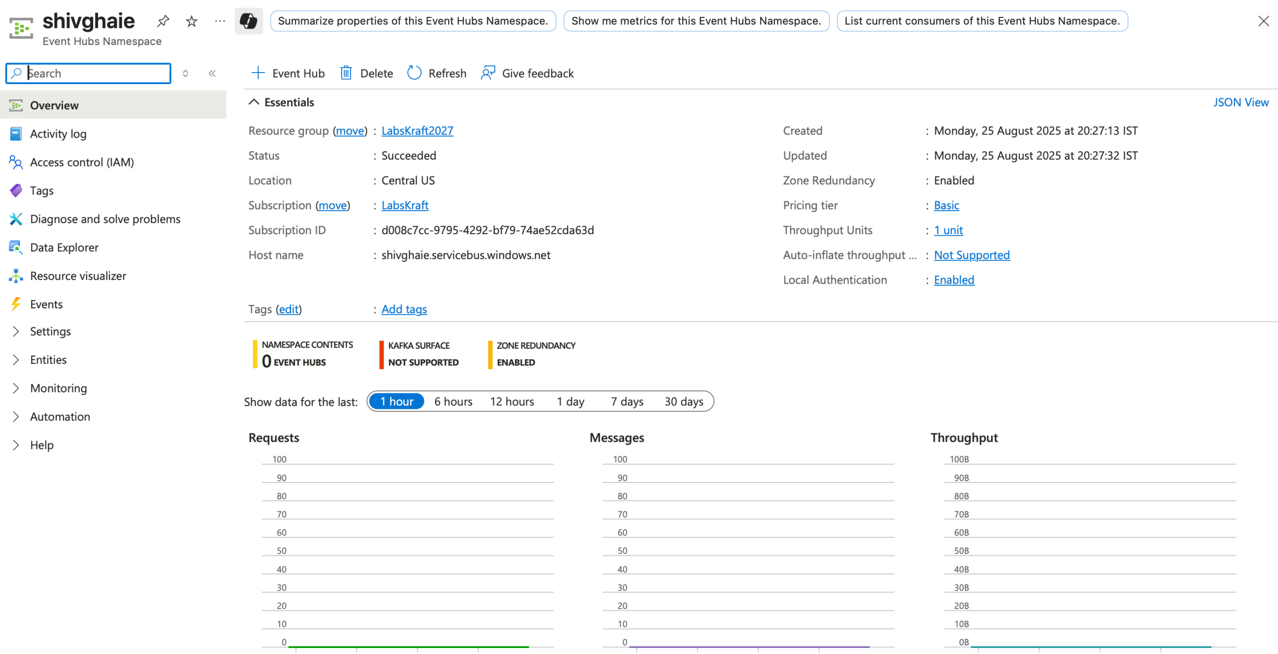


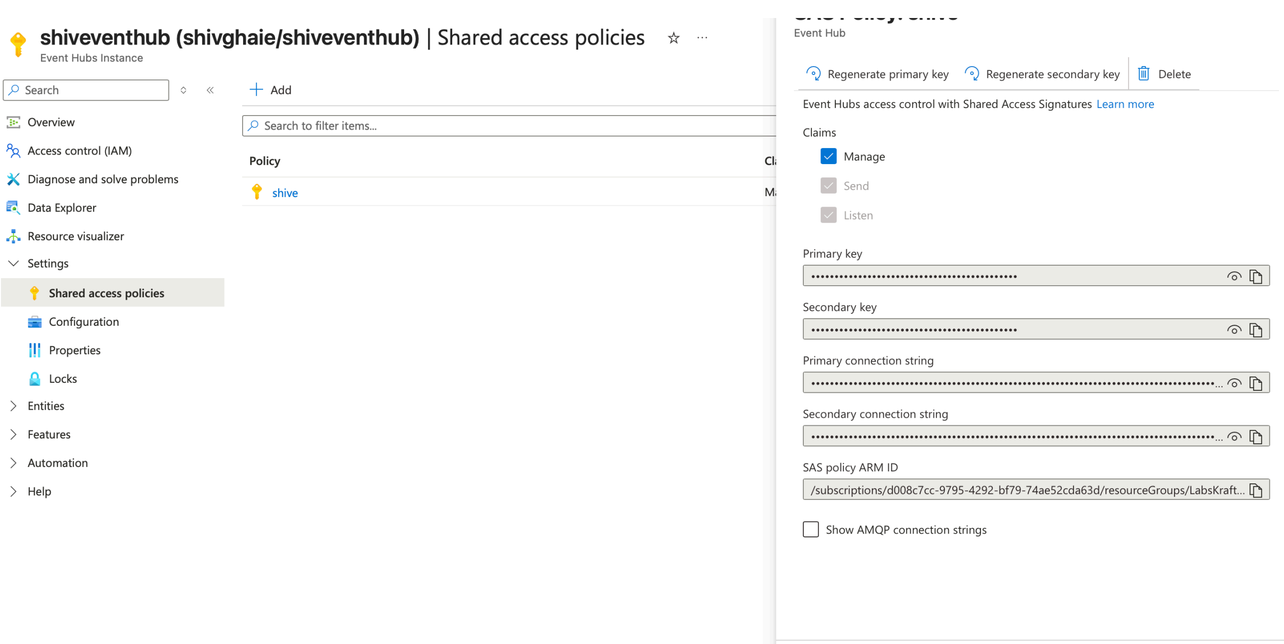


5. Creation of Namespace and Eventhub:

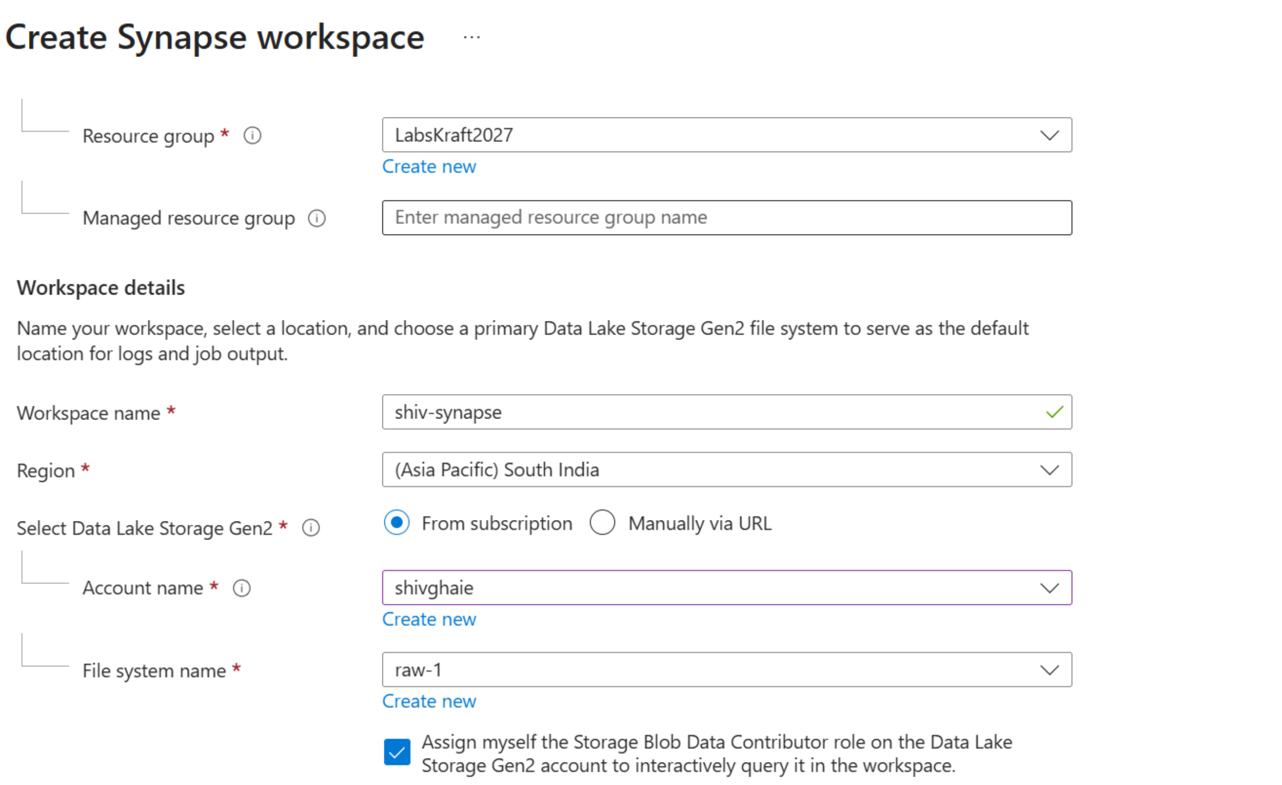


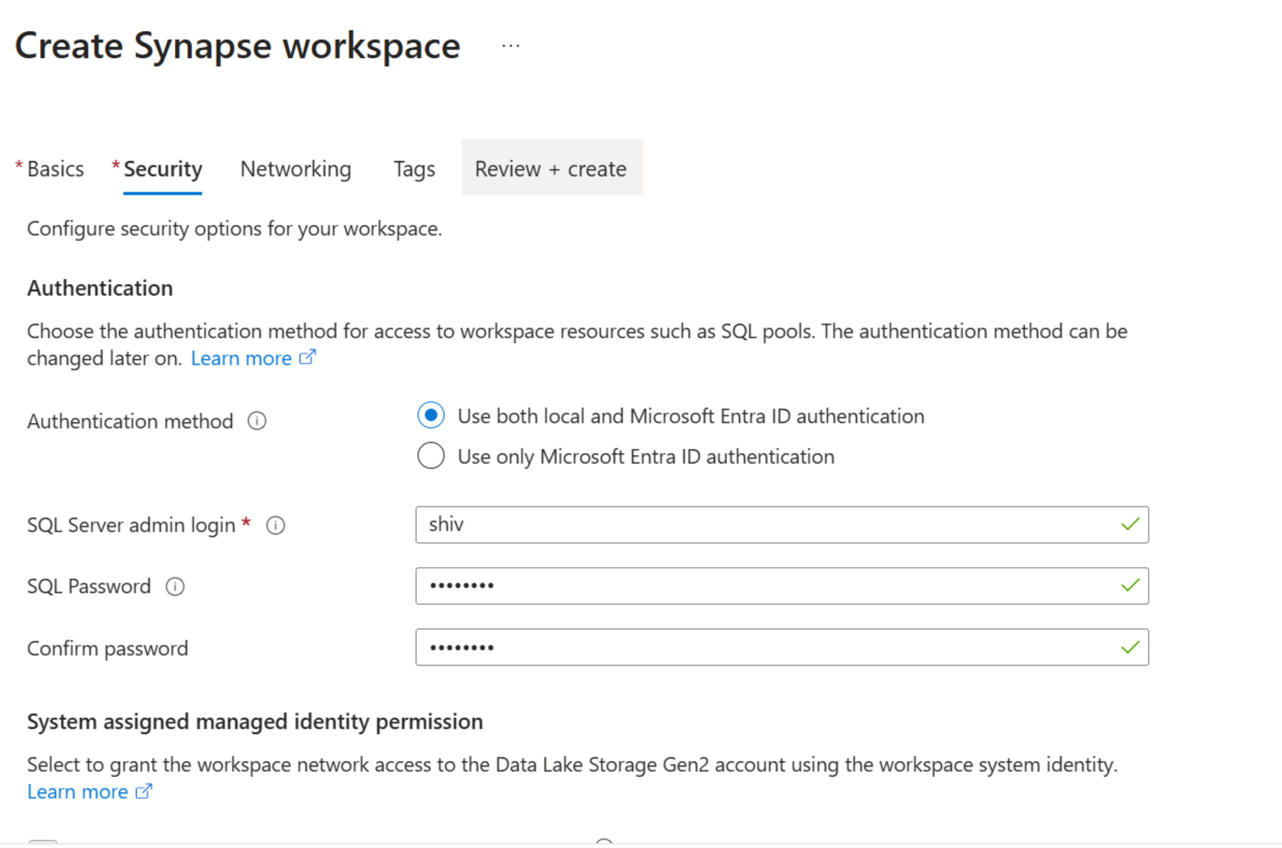

shiv-eventHub | Overview
Deployment
Search
Delete
Cancel
Redeploy
Download
Refresh
Overview
Inputs
Outputs
Template
Your deployment is complete
Deployment name : shiv-eventHub
Subscription : LabsKraft
Resource group : LabsKraft2027
Start time : 8/25/2025, 11:09:47 AM
Correlation ID : d97ddce0-b067-4cbd-8e40-e3...
Deployment details
Next steps
Go to resource

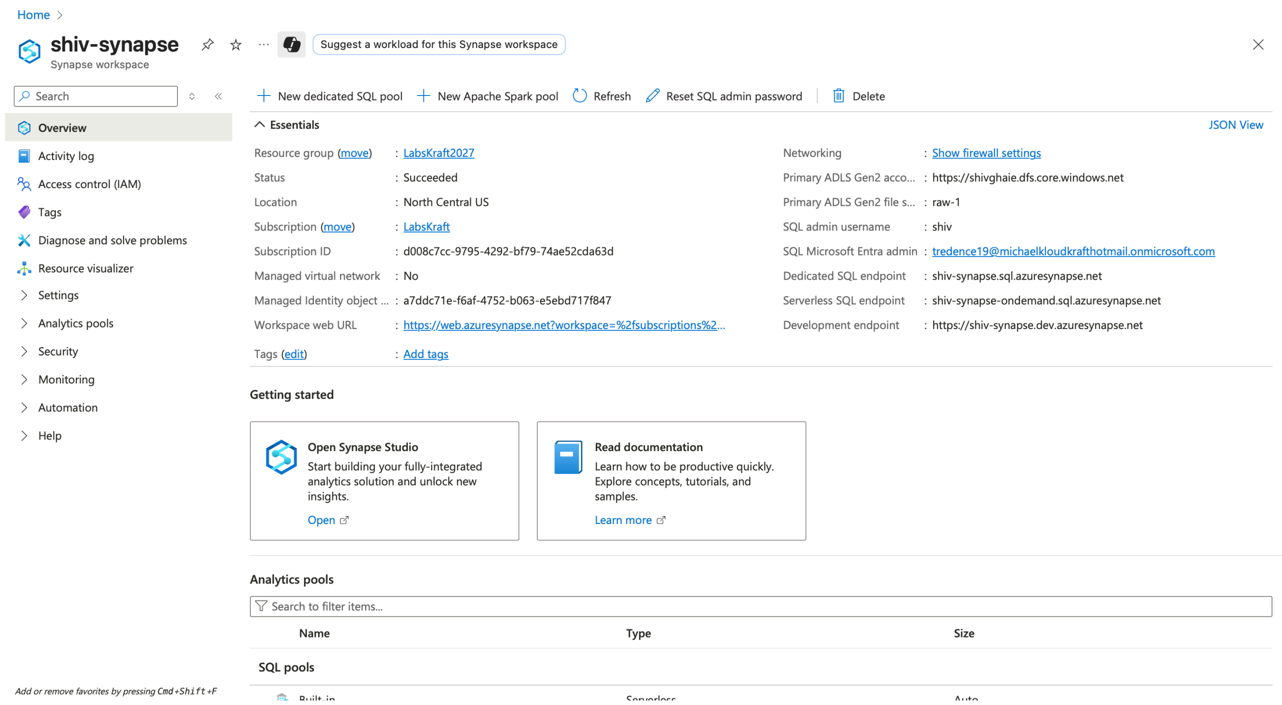

shivghaie
Event Hubs Namespace
Summarize properties of this Event Hubs Namespace.
Show me metrics for this Event Hubs Namespace.
List current consumers of this Event Hubs Namespace.
Search
Event Hub
Delete
Refresh
Give feedback
Overview
Activity log
Access control (IAM)
Tags
Diagnose and solve problems
Data Explorer
Resource visualizer
Events
Settings
Entities
Monitoring
Automation
Help
Essentials
JSON View
Resource group (move) : LabsKraft2027
Status : Succeeded
Location : Central US
Subscription (move) : LabsKraft
Subscription ID : d008c7cc-9795-4292-bf79-74ae52cda63d
Host name : shivghaie.servicebus.windows.net
Tags (edit) : Add tags
Created : Monday, 25 August 2025 at 20:27:13 IST
Updated : Monday, 25 August 2025 at 20:27:32 IST
Zone Redundancy : Enabled
Pricing tier : Basic
Throughput Units : 1 unit
Auto-inflate throughput ... : Not Supported
Local Authentication : Enabled
NAMESPACE CONTENTS
0 EVENT HUBS
KAFKA SURFACE
NOT SUPPORTED
ZONE REDUNDANCY
ENABLED
Show data for the last:
1 hour
6 hours
12 hours
1 day
7 days
30 days
Requests
Messages
Throughput


shiveventhub (shivghaie/shiveventhub) | Shared access policies
Event Hubs Instance
Search
Add
Search to filter items...
Overview
Access control (IAM)
Diagnose and solve problems
Data Explorer
Resource visualizer
Settings
Shared access policies
Configuration
Properties
Locks
Entities
Features
Automation
Help
Policy
shive
Event Hub
Regenerate primary key
Regenerate secondary key
Delete
Event Hubs access control with Shared Access Signatures Learn more
Claims
Manage
Send
Listen
Primary key
Secondary key
Primary connection string
Secondary connection string
SAS policy ARM ID
/subscriptions/d008c7cc-9795-4292-bf79-74ae52cda63d/resourceGroups/LabsKraft...
Show AMQP connection strings

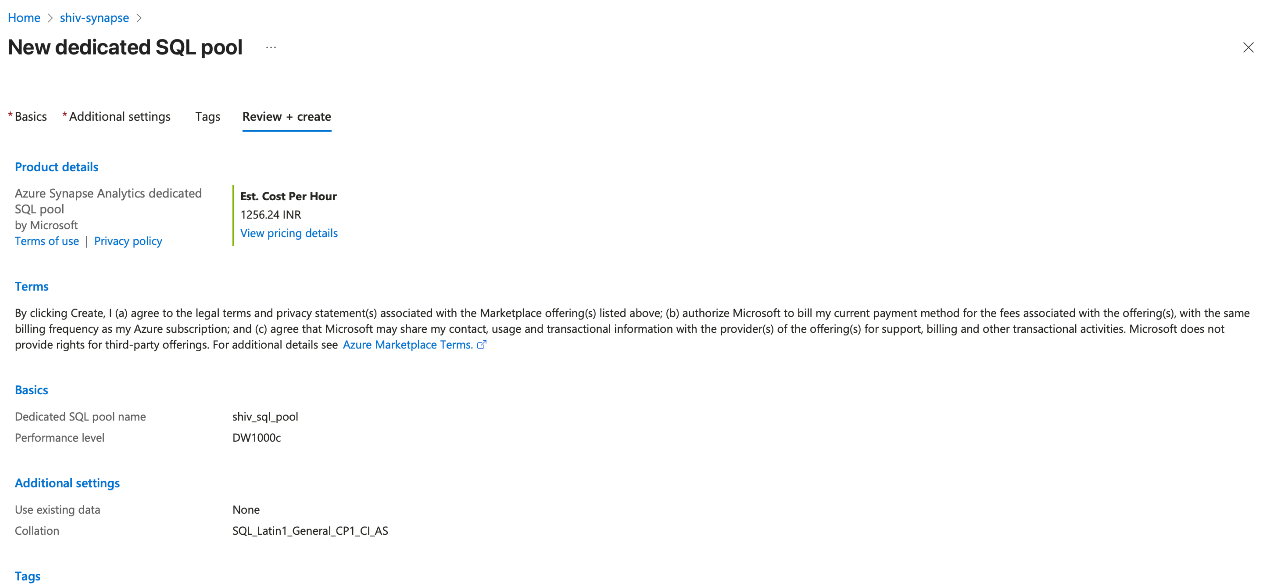
6. Creating Synapse Workspace:







7. Creation of new Dedicated SQL Pool:



Ingestion Mode

1. ADLS

Made a new Notebook in my workspace and wrote code for Medallion Architecture

(i) Schema

```sql
%sql
USE CATALOG cap1_shiv_catalog;
```

```sql
%sql
CREATE SCHEMA IF NOT EXISTS retail_demo
COMMENT 'Schema for BI Integration demo with DLT';
```

```sql
%sql
CREATE SCHEMA IF NOT EXISTS bronze
MANAGED LOCATION 'abfss://raw-1@shivghaie.dfs.core.windows.net/shiv_bronze/';

CREATE SCHEMA IF NOT EXISTS silver
MANAGED LOCATION 'abfss://raw-1@shivghaie.dfs.core.windows.net/shiv_silver/';

CREATE SCHEMA IF NOT EXISTS gold
MANAGED LOCATION 'abfss://raw-1@shivghaie.dfs.core.windows.net/shiv_gold/';
```

(ii) Bronze Layer

```python
from dlt import table
import pyspark.sql.functions as F
from pyspark.sql.types import StructType, StructField, StringType, DoubleType, IntegerType,
TimestampType
# :white_check_mark: Define schema for retail IoT CSV
iot_schema = StructType([
    StructField("OrderID", StringType(), True),
    StructField("CustomerID", StringType(), True),
    StructField("Region", StringType(), True),
    StructField("StoreID", StringType(), True),
    StructField("Product", StringType(), True),
    StructField("Quantity", IntegerType(), True),
    StructField("UnitPrice", DoubleType(), True),
    StructField("SalesAmount", DoubleType(), True),
    StructField("OrderTime", StringType(), True),
```

```
    StructField("DeviceType", StringType(), True),
    StructField("Temperature", DoubleType(), True),
    StructField("DeviceStatus", StringType(), True),
    StructField("AnomalyFlag", IntegerType(), True),
    StructField("AnomalyType", StringType(), True)
])
# :white_check_mark: Paths (use subdirectories to avoid overlap with DLT system files)
raw_path = "abfss://raw-1@shivghaie.dfs.core.windows.net/raw_data/"
schema_path = "abfss://raw-1@shivghaie.dfs.core.windows.net/dlt_schemas/bronze_orders"
checkpoint_path = "abfss://raw-1@shivghaie.dfs.core.windows.net/dlt_checkpoints/bronze_orders"
@table(
    name="cap1_shiv_catalog.capstone_retail_shivankur.bronze_orders",
    comment="Bronze table - raw ingested IoT retail data via Auto Loader"
)
def bronze_orders():
    return (
        spark.readStream
            .format("cloudFiles")
            .option("cloudFiles.format", "csv")
            .option("header", True)
            .option("cloudFiles.schemaLocation", schema_path)
            .schema(iot_schema)   # :white_check_mark: manually set schema (no infer needed)
            .load(raw_path)       # :white_check_mark: points to /raw_data/ folder
    )
```

(iii) Silver Layer

```
# Silver Layer - Cleaned IoT Orders
@table(
    name="capstone_retail_shivankur.silver_orders",
    comment="Silver layer: cleaned and deduplicated IoT orders with anomaly handling"
)
def silver_orders():
    # Read Bronze
    orders = dlt.read("cap1_shiv_catalog.capstone_retail_shivankur.bronze_orders")
    # Standardize schema
    orders = (
        orders.withColumn("Quantity", F.col("Quantity").cast(IntegerType()))
            .withColumn("UnitPrice", F.col("UnitPrice").cast(DoubleType()))
            .withColumn("SalesAmount", F.col("SalesAmount").cast(DoubleType()))
            .withColumn("Temperature", F.col("Temperature").cast(DoubleType()))
            .withColumn("OrderTime", F.to_timestamp("OrderTime", "M/d/yyyy H:mm"))
    )
    # Deduplicate by OrderID
    orders = orders.dropDuplicates(["OrderID"])
    # Handle missing anomaly type
    orders = orders.withColumn(
        "AnomalyType",
        F.when((F.col("AnomalyFlag") == 1) & (F.col("AnomalyType").isNull()), "Unknown")
         .when((F.col("AnomalyFlag") == 1) & (F.col("AnomalyType") == ""), "Unknown")
```

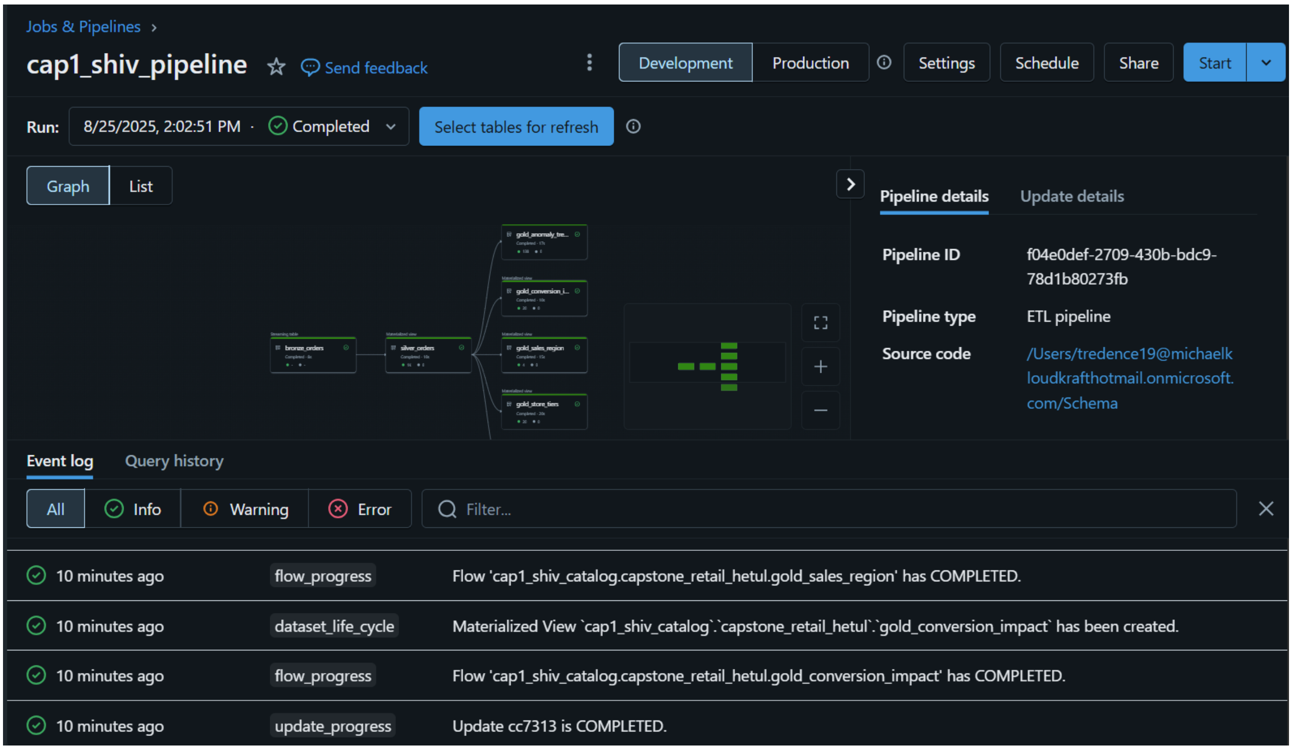
```
        .otherwise(F.col("AnomalyType"))
    )
    # Add business rule fields
    orders = (
        orders.withColumn("IsAnomaly", F.col("AnomalyFlag") == 1)
              .withColumn("DeviceStatus", F.upper(F.col("DeviceStatus")))
              .withColumn("NormalizedStoreID", F.upper(F.col("StoreID")))
              .withColumn("NormalizedProduct", F.upper(F.col("Product")))
    )
    return orders
```

(iv) Gold Layer

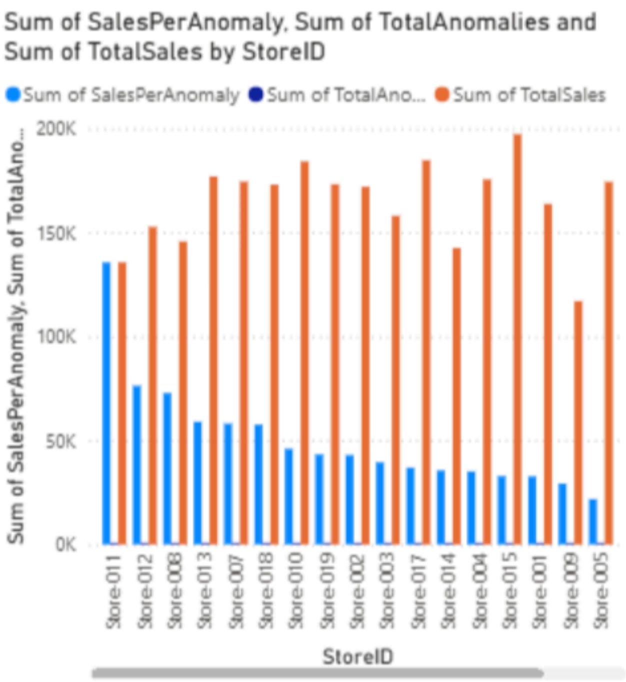
```
# Gold Layer Tables (capstone_retail.gold)

# 1. Daily / Monthly Sales per Region
@table(
    name="cap1_shiv_catalog.capstone_retail_shivankur.gold_sales_region",
    comment="Daily & Monthly Sales per Region"
)
def gold_sales_region():
    return (
        dlt.read("cap1_shiv_catalog.capstone_retail_shivankur.silver_orders")
        .withColumn("OrderDate", F.to_date("OrderTime", "M/d/yyyy H:mm"))
        .withColumn("YearMonth", F.date_format("OrderDate", "yyyy-MM"))
        .groupBy("Region", "OrderDate", "YearMonth")
        .agg(F.sum("SalesAmount").alias("TotalSales"))
    )
# 2. Device Anomaly Trend per Store
@table(
    name="cap1_shiv_catalog.capstone_retail_shivankur.gold_anomaly_trends",
    comment="Device Anomaly Trend per Store"
)
def gold_anomaly_trends():
    return (
        dlt.read("cap1_shiv_catalog.capstone_retail_shivankur.silver_orders")
        .withColumn("OrderDate", F.to_date("OrderTime", "M/d/yyyy H:mm"))
        .groupBy("StoreID", "OrderDate", "DeviceType", "AnomalyType")
        .agg(F.count("*").alias("AnomalyCount"))
    )
# 3. Conversion Rate Impact when devices fail
@table(
    name="cap1_shiv_catalog.capstone_retail_shivankur.gold_conversion_impact",
    comment="Conversion rate impact of device failures"
)
def gold_conversion_impact():
    df = dlt.read("cap1_shiv_catalog.capstone_retail_shivankur.silver_orders")
    total_orders = df.groupBy("StoreID").agg(F.count("*").alias("TotalOrders"))
    failed_orders = (
        df.filter(df.AnomalyFlag == 1)
```

```python
        .groupBy("StoreID")
        .agg(F.count("*").alias("FailedOrders"))
    )
    return (
        total_orders.join(failed_orders, "StoreID", "left")
        .withColumn("FailedOrders", F.coalesce(F.col("FailedOrders"), F.lit(0)))
        .withColumn("FailureRate", F.col("FailedOrders") / F.col("TotalOrders"))
    )
# 4. Store Tier Classification
@table(
    name="cap1_shiv_catalog.capstone_retail_shivankur.gold_store_tiers",
    comment="Store tiers based on total sales (High / Medium / Low performers)"
)
def gold_store_tiers():
    df = (
        dlt.read("cap1_shiv_catalog.capstone_retail_shivankur.silver_orders")
        .groupBy("StoreID")
        .agg(F.sum("SalesAmount").alias("TotalSales"))
    )
    return (
        df.withColumn(
            "StoreTier",
            F.when(F.col("TotalSales") > 100000, "High")
             .when(F.col("TotalSales") > 50000, "Medium")
             .otherwise("Low")
        )
    )
# 5. Weighted Average Sales vs Anomalies
@table(
    name="cap1_shiv_catalog.capstone_retail_shivankur.gold_weighted_sales_anomalies",
    comment="Weighted average of sales vs anomalies per store"
)
def gold_weighted_sales_anomalies():
    df = dlt.read("cap1_shiv_catalog.capstone_retail_shivankur.silver_orders")
    return (
        df.groupBy("StoreID")
        .agg(
            F.sum("SalesAmount").alias("TotalSales"),
            F.sum(F.when(F.col("AnomalyFlag") == 1, 1).otherwise(0)).alias("TotalAnomalies")
        )
        .withColumn("SalesPerAnomaly",
            F.when(F.col("TotalAnomalies") > 0, F.col("TotalSales") / F.col("TotalAnomalies"))
             .otherwise(F.lit(None))
        )
    )
```

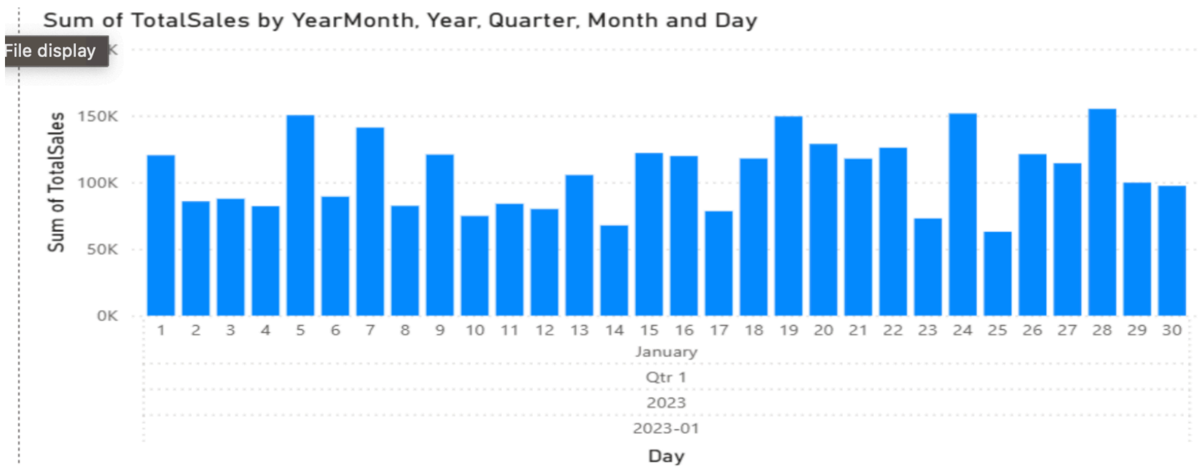
Final Pipeline



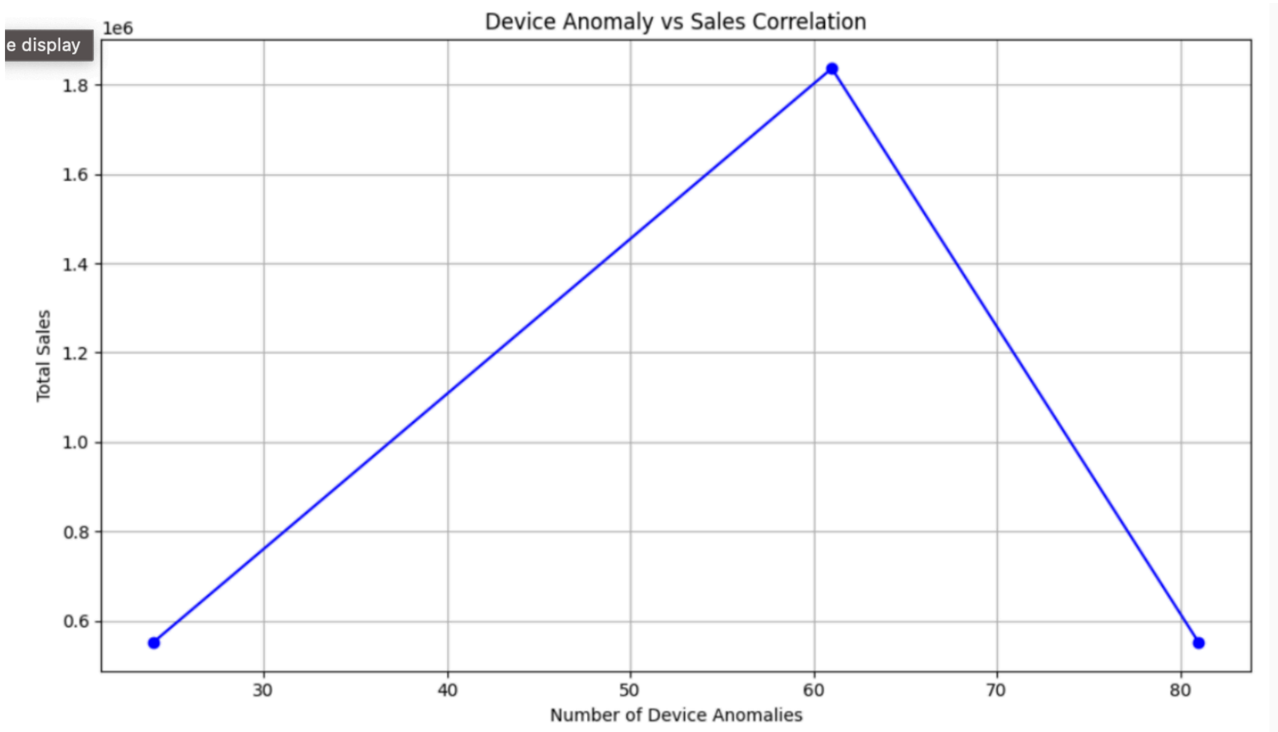
Power BI



StoreID	StoreTier	Sum of TotalSales
Store-001	High	1,63,738.09
Store-002	High	1,71,989.14
Store-003	High	1,58,060.91
Store-004	High	1,75,558.23
Store-005	High	1,74,334.11
Store-006	High	1,47,107.99
Store-007	High	1,74,436.47
Store-008	High	1,45,685.27
Store-009	High	1,17,012.92
Store-010	High	1,84,143.77
Store-011	High	1,35,648.97
Store-012	High	1,52,687.06
Store-013	High	1,76,925.59
Store-014	High	1,42,545.46
Store-015	High	1,97,270.28
Store-016	High	1,74,728.79
Store-017	High	1,84,736.13
Store-018	High	1,73,029.89
Store-019	High	1,73,193.32
Store-020	Medium	83,245.81
Total		**32,06,078.20**



Sum of TotalSales by YearMonth, Year, Quarter, Month and Day
File display
Sum of TotalSales
150K
100K
50K
0K
1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30
January
Qtr 1
2023
2023-01
Day



Device Anomaly vs Sales Correlation
e display
1e6
1.8
1.6
1.4
1.2
1.0
0.8
0.6
Total Sales
30 40 50 60 70 80
Number of Device Anomalies

2. Event Hub

Code:

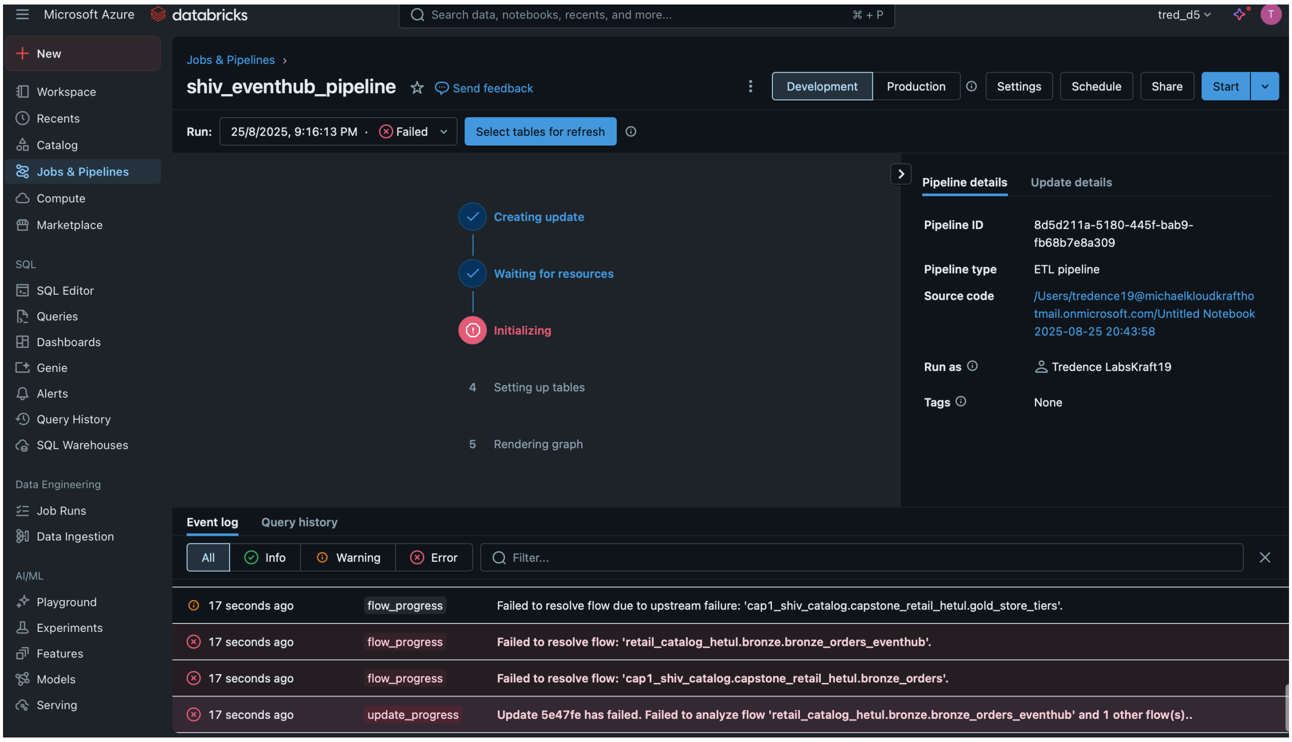
```
# ==========================================
# Bronze → Silver → Gold with Event Hub Integration
# ==========================================
from dlt import table, read
import pyspark.sql.functions as F
from pyspark.sql.types import *
# ================================
# :one: Bronze Layer: Schema & CSV
# ================================
iot_schema = StructType([
  StructField("OrderID", StringType(), True),
  StructField("CustomerID", StringType(), True),
  StructField("Region", StringType(), True),
  StructField("StoreID", StringType(), True),
  StructField("Product", StringType(), True),
  StructField("Quantity", IntegerType(), True),
  StructField("UnitPrice", DoubleType(), True),
  StructField("SalesAmount", DoubleType(), True),
  StructField("OrderTime", StringType(), True),
  StructField("DeviceType", StringType(), True),
  StructField("Temperature", DoubleType(), True),
  StructField("DeviceStatus", StringType(), True),
  StructField("AnomalyFlag", IntegerType(), True),
  StructField("AnomalyType", StringType(), True)
])
raw_path = "abfss://raw-1@shivghaie.dfs.core.windows.net/incoming/"
schema_path = "abfss://raw-1@shivghaie.dfs.core.windows.net/dlt_schemas/bronze_orders"
@table(
  name="retail_catalog_shivankur.bronze.bronze_orders_csv",
  comment="Bronze table - raw IoT CSV"
)
def bronze_orders_csv():
   return (
    spark.readStream
      .format("cloudFiles")
      .option("cloudFiles.format", "csv")
      .option("header", True)
      .option("cloudFiles.schemaLocation", schema_path)
      .schema(iot_schema)
      .load(raw_path)
      .withColumn("OrderTime", F.to_timestamp("OrderTime"))
  )
# ================================
# :two: Bronze Layer: Event Hub IoT
# ================================
eventhub_connection_string = “”
```

```
consumer_group = "$Default"
ehConf = {
   'eventhubs.connectionString': eventhub_connection_string,
   'eventhubs.consumerGroup': consumer_group,
   'eventhubs.startingPosition': '@latest'
}
@table(
  name="retail_catalog_shivankur.bronze.bronze_orders_eventhub",
  comment="Bronze table - live IoT data from Event Hub"
)
def bronze_orders_eventhub():
  df = spark.readStream \
    .format("eventhubs") \
    .options(**ehConf) \
    .load()
  # Event Hub body is bytes → convert to string → parse JSON
  df1 = df.selectExpr("cast(body as string) as json_string")
  df_parsed = df1.select(F.from_json(F.col("json_string"), iot_schema).alias("data")).select("data.*")
  return df_parsed.withColumn("OrderTime", F.to_timestamp("OrderTime"))
# ===================================
# :three: Unified Bronze Table
# ===================================
@table(
  name="cap1_shiv_catalog.capstone_retail_shivankur.bronze_orders",
  comment="Unified Bronze table (CSV + Event Hub)"
)
def bronze_orders():
  df_csv = read("cap1_shiv_catalog.capstone_retail_shivankur.bronze_orders_csv")
  df_eh = read("cap1_shiv_catalog.capstone_retail_shivankur.bronze_orders_eventhub")
  return df_csv.unionByName(df_eh)
# ===================================
# :four: Silver Layer
# ===================================
@table(
  name="cap1_shiv_catalog.capstone_retail_shivankur.silver_orders",
  comment="Silver layer: cleaned and deduplicated IoT orders with anomaly handling"
)
def silver_orders():
  orders = read("cap1_shiv_catalog.capstone_retail_shivankur.bronze_orders")
  orders = (
    orders.withColumn("Quantity", F.col("Quantity").cast(IntegerType()))
       .withColumn("UnitPrice", F.col("UnitPrice").cast(DoubleType()))
       .withColumn("SalesAmount", F.col("SalesAmount").cast(DoubleType()))
       .withColumn("Temperature", F.col("Temperature").cast(DoubleType()))
       .withColumn("OrderTime", F.to_timestamp("OrderTime", "M/d/yyyy H:mm"))
  )
  orders = orders.dropDuplicates(["OrderID"])
  orders = orders.withColumn(
     "AnomalyType",
    F.when((F.col("AnomalyFlag") == 1) & (F.col("AnomalyType").isNull()), "Unknown")
     .when((F.col("AnomalyFlag") == 1) & (F.col("AnomalyType") == ""), "Unknown")
```

```
    .otherwise(F.col("AnomalyType"))
  )
  orders = (
    orders.withColumn("IsAnomaly", F.col("AnomalyFlag") == 1)
       .withColumn("DeviceStatus", F.upper(F.col("DeviceStatus")))
       .withColumn("NormalizedStoreID", F.upper(F.col("StoreID")))
       .withColumn("NormalizedProduct", F.upper(F.col("Product")))
  )
  return orders
# ==================================
# :five: Gold Layer
# ==================================
@table(
  name="cap1_shiv_catalog.capstone_retail_shivankur.gold_sales_region",
  comment="Daily & Monthly Sales per Region"
)
def gold_sales_region():
   return (
    read("cap1_shiv_catalog.capstone_retail_shivankur.silver_orders")
    .withColumn("OrderDate", F.to_date("OrderTime", "M/d/yyyy H:mm"))
    .withColumn("YearMonth", F.date_format("OrderDate", "yyyy-MM"))
    .groupBy("Region", "OrderDate", "YearMonth")
    .agg(F.sum("SalesAmount").alias("TotalSales"))
  )
@table(
  name="cap1_shiv_catalog.capstone_retail_shivankur.gold_anomaly_trends",
  comment="Device Anomaly Trend per Store"
)
def gold_anomaly_trends():
   return (
    read("cap1_shiv_catalog.capstone_retail_shivankur.silver_orders")
    .withColumn("OrderDate", F.to_date("OrderTime", "M/d/yyyy H:mm"))
    .groupBy("StoreID", "OrderDate", "DeviceType", "AnomalyType")
    .agg(F.count("*").alias("AnomalyCount"))
  )
@table(
  name="cap1_shiv_catalog.capstone_retail_shivankur.gold_conversion_impact",
  comment="Conversion rate impact of device failures"
)
def gold_conversion_impact():
  df = read("cap1_shiv_catalog.capstone_retail_shivankur.silver_orders")
  total_orders = df.groupBy("StoreID").agg(F.count("*").alias("TotalOrders"))
  failed_orders = (
    df.filter(df.AnomalyFlag == 1)
     .groupBy("StoreID")
     .agg(F.count("*").alias("FailedOrders"))
  )
  return (
    total_orders.join(failed_orders, "StoreID", "left")
    .withColumn("FailedOrders", F.coalesce(F.col("FailedOrders"), F.lit(0)))
    .withColumn("FailureRate", F.col("FailedOrders") / F.col("TotalOrders"))
  )
```

```
@table(
  name="cap1_shiv_catalog.capstone_retail_shivankur.gold_store_tiers",
  comment="Store tiers based on total sales (High / Medium / Low performers)"
)
def gold_store_tiers():
  df = (
    read("cap1_shiv_catalog.capstone_retail_shivankur.silver_orders")
    .groupBy("StoreID")
    .agg(F.sum("SalesAmount").alias("TotalSales"))
  )
  return (
    df.withColumn(
      "StoreTier",
      F.when(F.col("TotalSales") > 100000, "High")
       .when(F.col("TotalSales") > 50000, "Medium")
       .otherwise("Low")
    )
  )
@table(
  name="cap1_shiv_catalog.capstone_retail_shivankur.gold_weighted_sales_anomalies",
  comment="Weighted average of sales vs anomalies per store"
)
def gold_weighted_sales_anomalies():
  df = read("cap1_shiv_catalog.capstone_retail_shivankur.silver_orders")
  return (
    df.groupBy("StoreID")
    .agg(
      F.sum("SalesAmount").alias("TotalSales"),
      F.sum(F.when(F.col("AnomalyFlag") == 1, 1).otherwise(0)).alias("TotalAnomalies")
    )
    .withColumn("SalesPerAnomaly",
      F.when(F.col("TotalAnomalies") > 0, F.col("TotalSales") / F.col("TotalAnomalies"))
       .otherwise(F.lit(None))
    )
  )
```

Final Output:


Microsoft Azure
databricks
Search data, notebooks, recents, and more...
tred_d5
New
Workspace
Recents
Catalog
Jobs & Pipelines
Compute
Marketplace
SQL
SQL Editor
Queries
Dashboards
Genie
Alerts
Query History
SQL Warehouses
Data Engineering
Job Runs
Data Ingestion
AI/ML
Playground
Experiments
Features
Models
Serving
Jobs & Pipelines
shiv_eventhub_pipeline
Send feedback
Development
Production
Settings
Schedule
Share
Start
Run: 25/8/2025, 9:16:13 PM · Failed
Select tables for refresh
Creating update
Waiting for resources
Initializing
4 Setting up tables
5 Rendering graph
Pipeline details
Update details
Pipeline ID 8d5d211a-5180-445f-bab9-fb68b7e8a309
Pipeline type ETL pipeline
Source code /Users/tredence19@michaelkloudkrafthotmail.onmicrosoft.com/Untitled Notebook 2025-08-25 20:43:58
Run as Tredence LabsKraft19
Tags None
Event log
Query history
All
Info
Warning
Error
Filter...
17 seconds ago flow_progress Failed to resolve flow due to upstream failure: 'cap1_shiv_catalog.capstone_retail_hetul.gold_store_tiers'.
17 seconds ago flow_progress Failed to resolve flow: 'retail_catalog_hetul.bronze.bronze_orders_eventhub'.
17 seconds ago flow_progress Failed to resolve flow: 'cap1_shiv_catalog.capstone_retail_hetul.bronze_orders'.
17 seconds ago update_progress Update 5e47fe has failed. Failed to analyze flow 'retail_catalog_hetul.bronze.bronze_orders_eventhub' and 1 other flow(s)..

Terminal

nano iot_stream.py

```
GNU nano 7.2                                                    iot_stream.py
import random
import time
import uuid
from datetime import datetime
from azure.eventhub import EventHubProducerClient, EventData
# -----------------------------
# CONFIGURATION
# -----------------------------
EVENT_HUB_CONNECTION_STR = "Endpoint=sb://shivghaie.servicebus.windows.net/;SharedAccessKeyName=shive;SharedAccessKey=sVW+9iciTHp2scj66h8oDqa+WIf5jy0fE+AEhCVRSn4=;EntityPath=shiveventhub"
EVENT_HUB_NAME = "shiveventHub"
# -----------------------------
# Function to generate one IoT row
# -----------------------------
def generate_row():
   quantity = random.randint(1, 10)
   unit_price = round(random.uniform(50, 500), 2)
   sales_amount = round(quantity * unit_price, 2)
   device_status = random.choice(["Active", "Inactive", "Error"])
   anomaly_flag = 1 if device_status != "Active" else 0
   anomaly_type = random.choice(["Overheat","LowBattery","Disconnected"]) if anomaly_flag else "None"
   return {
       "OrderID": str(uuid.uuid4()),
       "CustomerID": f"CUST{random.randint(1000,9999)}",
       "Region": random.choice(["North","South","East","West"]),
       "StoreID": random.choice(["Store001","Store002","Store003"]),
       "Product": random.choice(["Laptop","Phone","Monitor","Keyboard","Mouse"]),
       "Quantity": quantity,
       "UnitPrice": unit_price,
       "SalesAmount": sales_amount,
       "OrderTime": datetime.now().strftime("%Y-%m-%d %H:%M:%S"),
       "DeviceType": random.choice(["SensorA","SensorB","SensorC"]),
       "Temperature": round(random.uniform(20, 100), 2),
       "DeviceStatus": device_status,
       "AnomalyFlag": anomaly_flag,
       "AnomalyType": anomaly_type
   }
# -----------------------------
# Event Hub Producer
# -----------------------------
producer = EventHubProducerClient.from_connection_string(
   conn_str=EVENT_HUB_CONNECTION_STR,
   eventhub_name=EVENT_HUB_NAME
)
# -----------------------------
# Send data every second
# -----------------------------
try:
   for _ in range(1000): # Send exactly 1000 rows
       row = generate_row()
       event_data = EventData(str(row)) # convert dict to string
       with producer:
           producer.send_batch([event_data])
       print(f"Sent row: {row}")
       time.sleep(1) # 1-second interval
except KeyboardInterrupt:
   print("Streaming stopped by user.")
finally:
   producer.close()
```